

15-Minute Neighborhood Score

A console application that scores how walkable a neighborhood is and helps residents make more sustainable housing decisions.



AGENDA

What We'll Cover

1

The Problem

Why car-dependent neighborhoods are a sustainability issue

3

Design & Architecture

Flowchart, program structure, two-file layout

5

Testing

Real scenarios, real output, a real bug we found and fixed

2

Our Solution

What the 15-Minute Neighborhood Score app does

4

Computational Thinking

Decomposition, abstraction, pattern recognition, algorithms

6

Discussion & Next Steps

Limitations, future work, conclusion

Why Proximity Matters

SDG 11 calls for inclusive, safe, and sustainable cities. Two of its targets are fundamentally about proximity: accessible transport (11.2) and universal access to green and public space (11.7). The 15-minute city concept turns that into a design standard: residents should reach daily essentials within a 15-minute walk.

Six essentials, one radius:



Grocery



Healthcare



Education



Transit



Parks



Retail



**15 MIN
WALK**

THE PROBLEM

No Easy Way to Answer: “Is This Neighborhood Actually Walkable?”

1

No single score

Residents can name the nearest shop, but rarely have one objective number covering every daily need at once.

2

No easy comparison

Choosing between two homes means comparing walkability by gut feeling, not data.

3

Tools are inaccessible

Real walkability tools are GIS software for planners, or web services needing accounts and live APIs.

Our question: can a resident, using nothing more than a basic Python program, turn what they already know about a neighborhood into an objective, comparable walkability score with actionable feedback?

SYSTEM OBJECTIVES

What the Application Must Do

- ✓ Record a neighborhood assessment: walking time to 6 essential categories
- ✓ Compute an objective 0-100 score with a clear rating band
- ✓ Persist assessments across sessions (JSON storage)
- ✓ Generate targeted recommendations for the weakest categories
- ✓ Support side-by-side comparison of two saved assessments
- ✓ Validate every input and never crash on bad data

2

files, no classes

Every function does one thing: take plain data in, return plain data out.

OUR SOLUTION

A Menu-Driven Console App

15-MINUTE NEIGHBORHOOD SCORE

1. Create New Assessment
2. View All Assessments
3. View Score Breakdown
4. View Recommendations
5. Compare Two Assessments
6. Edit an Assessment
7. Delete an Assessment
8. About SDG 11 & Methodology
0. Exit

1

Fast to use

Every action is one menu number away: create, view, compare, edit, delete.

2

Always valid

Every prompt retries on bad input instead of crashing.

3

Remembers you

Assessments persist to disk, so history builds up across sessions.

4

Actionable

Recommendations name the weakest categories and suggest real next steps.

Two Files, Plain Functions

main.py

The program you run

Prints the menu and every screen. Asks for input with simple while loops that keep re-asking until the answer is valid. One handler function per menu option.

neighborhood_score.py

The data and the math

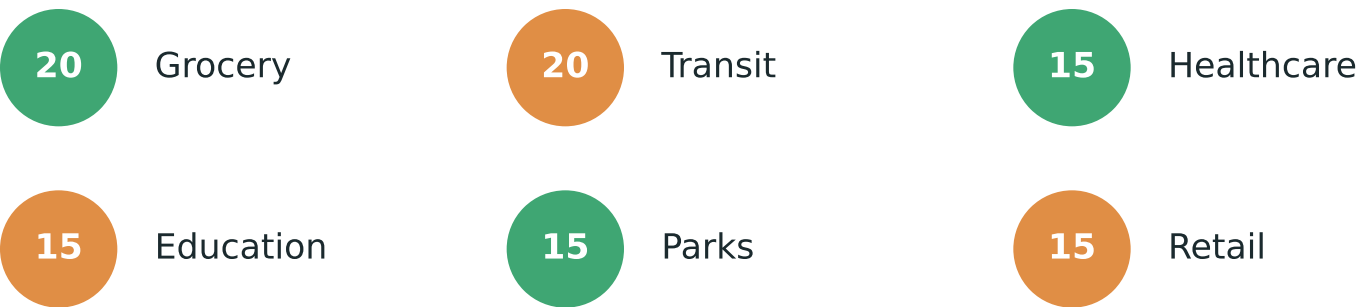
Holds the category weights and the walking-time table. Plain functions for scoring, recommendations, and saving and loading JSON. No classes, just functions and dictionaries.

One direction only: main.py imports neighborhood_score.py, never the other way around.

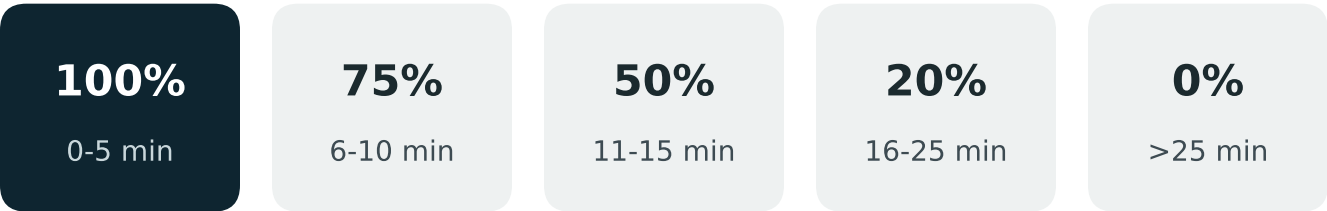
SCORING METHODOLOGY

From Minutes to a 0-100 Score

Six categories, weighted to 100:



`get_tier_percentage(minutes)` is one rule, reused for every category:



Total score =
 $\sum (\text{weight} \times \text{tier } \%)$

●	≥ 85	Excellent
●	70-84	Good
●	50-69	Fair
●	< 50	Poor

Program Structure

main.py — the console program

- print_header, print_message, print_separator: screen output
- get_menu_choice, get_nonempty_text, get_minutes: input, retries on bad input
- print_score_breakdown, print_comparison, print_recommendations: formatted output
- handle_new_assessment, handle_edit, handle_delete, ...: one handler per menu option

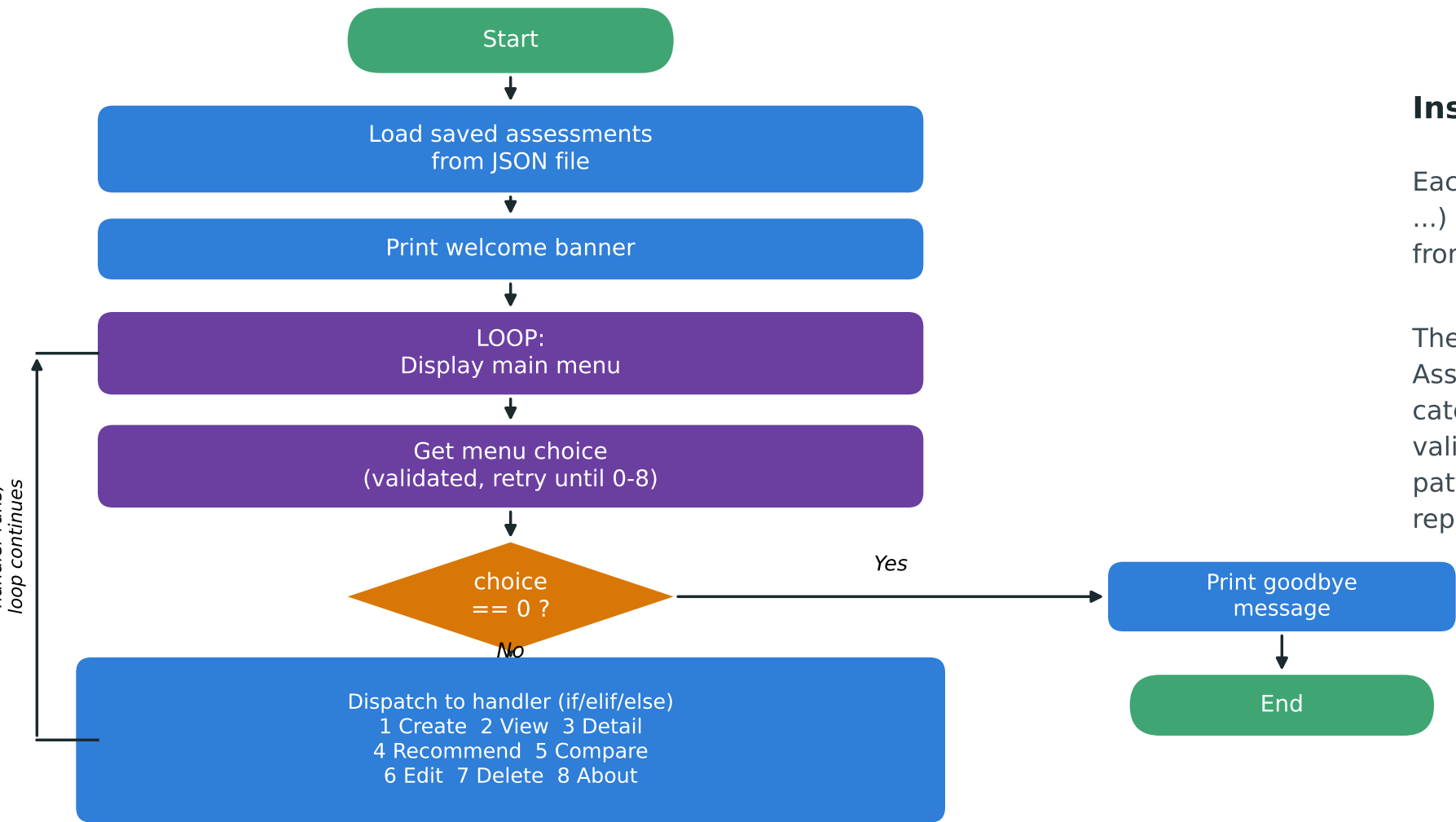
-
- main(): the while-loop menu

An assessment is just a dictionary, for example {"id": 1, "nickname": "Current Apartment", "category_times": {"grocery": 4, ...}}, saved straight to and read straight from JSON.

neighborhood_score.py — data and math

- CATEGORIES, TIER_TABLE, RATING_BANDS: constant lookup tables
- get_tier_percentage, calculate_total_score, classify_rating: scoring
- generate_recommendations, get_tip_for_category
- load_all_assessments, save_all_assessments, add_assessment, ...: JSON storage

Program Flowchart — Main Loop



Inside every handler:

Each menu option (Create, Edit, Compare, ...) is its own handler function, called from the dispatch step.

The richest example is Create New Assessment: a for-loop over the 6 categories, each wrapped in a while-retry validation loop. It's the same nested-loop pattern documented in full in the project report.

Real Program Output

SCORE BREAKDOWN: Current Apartment (ID 1)

Category	Minutes	Tier	Points
Grocery / Fresh Food Access	4.0	100%	20.0 / 20
Healthcare / Pharmacy	12.0	50%	7.5 / 15
Education / Childcare	20.0	20%	3.0 / 15
Public Transit Access	3.0	100%	20.0 / 20
Parks / Green Space	9.0	75%	11.3 / 15
Everyday Retail & Services	7.0	75%	11.3 / 15

TOTAL SCORE: 73.0 / 100 -> Rating: GOOD

1

Menu-driven

Every screen loops back to the menu; nothing dead-ends.

2

Validated

Bad input retries with a specific message, never crashes.

3

Persistent

JSON storage means history survives a relaunch.

Four Concepts, Applied Throughout



Decomposition

Two files, each split into small, single-purpose functions.



Abstraction

`calculate_total_score()` hides the tier lookup and weighting behind one function call.



Pattern Recognition

One tier function reused for all 6 categories, not duplicated 6x.



Algorithmic Thinking

Explicit accumulate and sort-then-slice algorithms in the scoring code.

TESTING

Tested for Real, Not Just Demoed

3

test runs

20+

scenarios covered

0

unhandled exceptions

A real bug, found and fixed

The first version of “Compare Two Assessments” embedded long nicknames directly into a fixed-width table column, so a 26-character name overran its column with no separating space. Fixed by switching to constant-width “Option A / B” headers, verified against a deliberately long 48-character nickname.

DISCUSSION

What Works, and What's Next

Strengths

- ✓ Objective, comparable 0-100 score
- ✓ Actionable, targeted recommendations
- ✓ Persistent history across sessions
- ✓ Zero external dependencies
- ✓ Robust input validation

Limitations & Future Work

- Self-reported walking times, not GPS
- Straight-line distance estimates, not real routes
- Fixed category weights for all users
- Measures presence, not safety or quality
- Single-user local JSON storage

CONCLUSION

A Practical Answer to a Real Question About Where to Live

Two files, no classes, and a tested, documented codebase: built to turn “is this neighborhood walkable?” from a gut feeling into an objective, comparable, actionable score that supports SDG 11's vision of accessible, sustainable cities.



Thank You

Questions & Live Demo

GitHub: github.com/DevFadul/15-minute-neighborhood-score